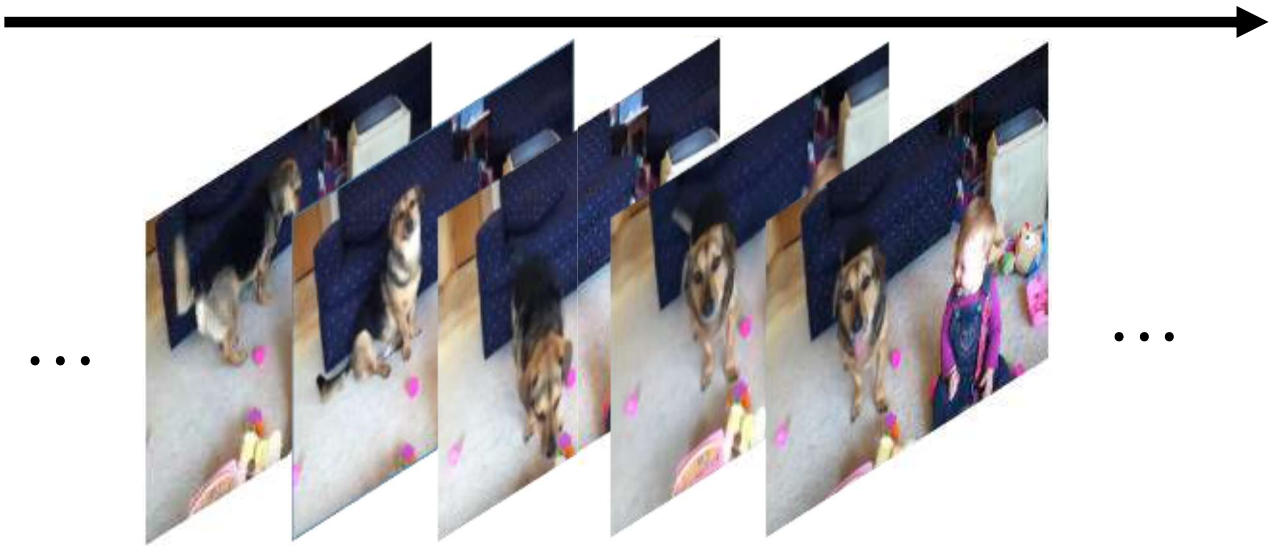
# Lecture 9 : Video Understanding

# Today: Video = 2D + Time

A video is a sequence of images
4D tensor: T x 3 x H x W



This image is CC0 public domain

# Example task: Video Classification



Input video:
T x 3 x H x W



Swimming
Running
Jumping
Eating
Standing

Running video is in the public domain

# Example task: Video Classification



Images: Recognize objects

Dog
Cat
Fish
Truck



Videos: Recognize actions

Swimming
Running
Jumping
Eating
Standing

Running video is in the public domain

# Problem: Videos are big!

Videos are often ~30 frames per second (fps)

Size of uncompressed video
(3 bytes per pixel):

SD (640 x 480): ~1.5 GB per minute
HD (1920 x 1080): ~10 GB per minute



Input video:
T x 3 x H x W

# Problem: Videos are big!



Input video:
T x 3 x H x W

Videos are often ~30 frames per second (fps)

Size of uncompressed video
(3 bytes per pixel):

SD (640 x 480): ~1.5 GB per minute
HD (1920 x 1080): ~10 GB per minute

One Solution: Train on short clips:
low fps and low spatial resolution
e.g. T = 16, H=W=112
(3.2 seconds at 5 fps, 588 KB)

# Training vs Testing on Video Clips

Raw video: Long, high FPS



Training: Train model to classify short clips with low FPS

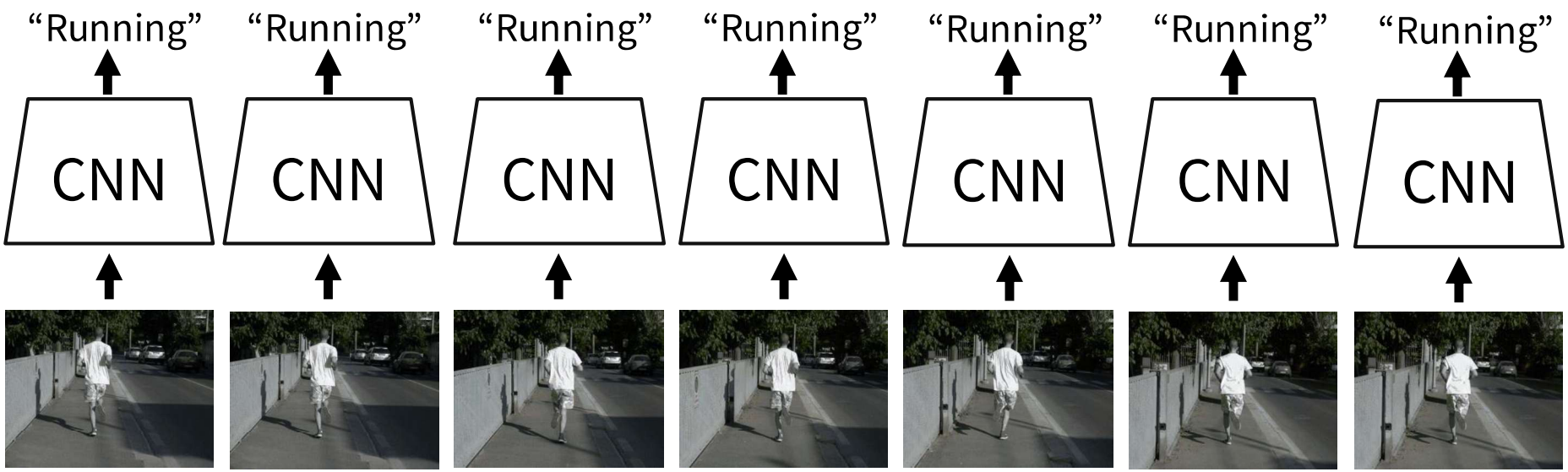


Testing: Run model on different clips, average predictions



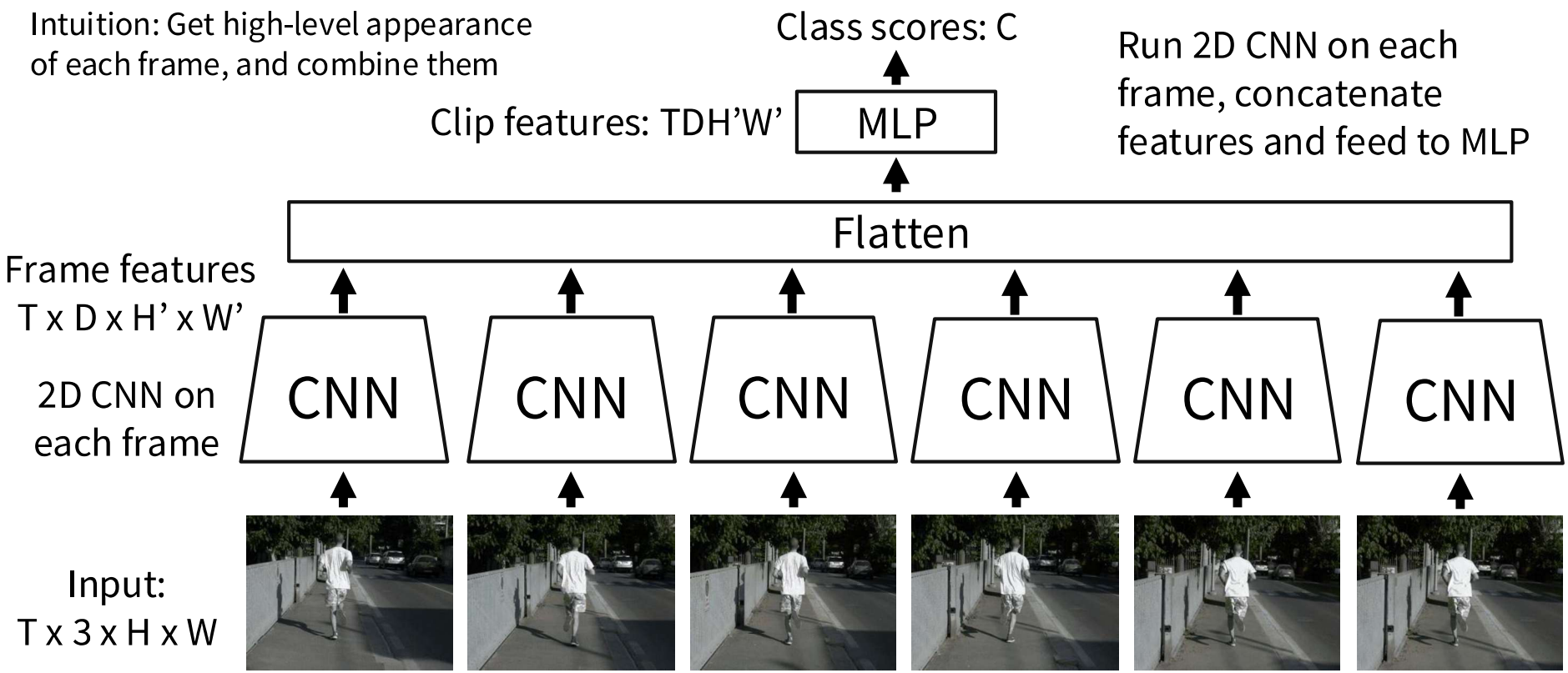
# Video Classification: Single-Frame CNN

**Simple idea:** train normal 2D CNN to classify video frames independently!
(Average predicted probs at test-time)
Often a very strong baseline for video classification



# Video Classification: Late Fusion (with FC layers)

Intuition: Get high-level appearance of each frame, and combine them




Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

# Video Classification: Late Fusion (with pooling)

Intuition: Get high-level appearance of each frame, and combine them

Class scores: C

Linear

Clip features: D

Run 2D CNN on each frame, pool features and feed to Linear

Average Pool over space and time

Frame features
T x D x H' x W'

2D CNN on each frame

CNN CNN CNN CNN CNN CNN

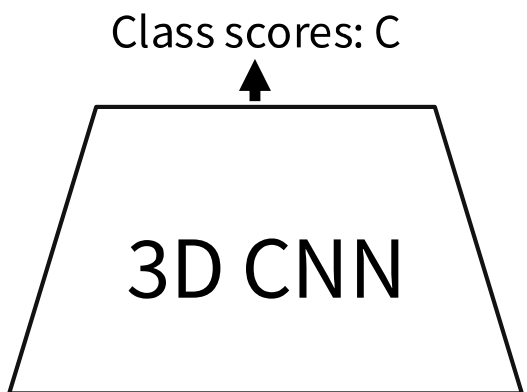
Input:
T x 3 x H x W



# Video Classification: Early Fusion with 3D CNNs

Intuition: Use 3D versions of convolution and pooling to slowly fuse temporal information over the course of the network

Each activation map in the network is a 4D tensor:
D x T x H x W
Use 3D conv and 3D pooling operations

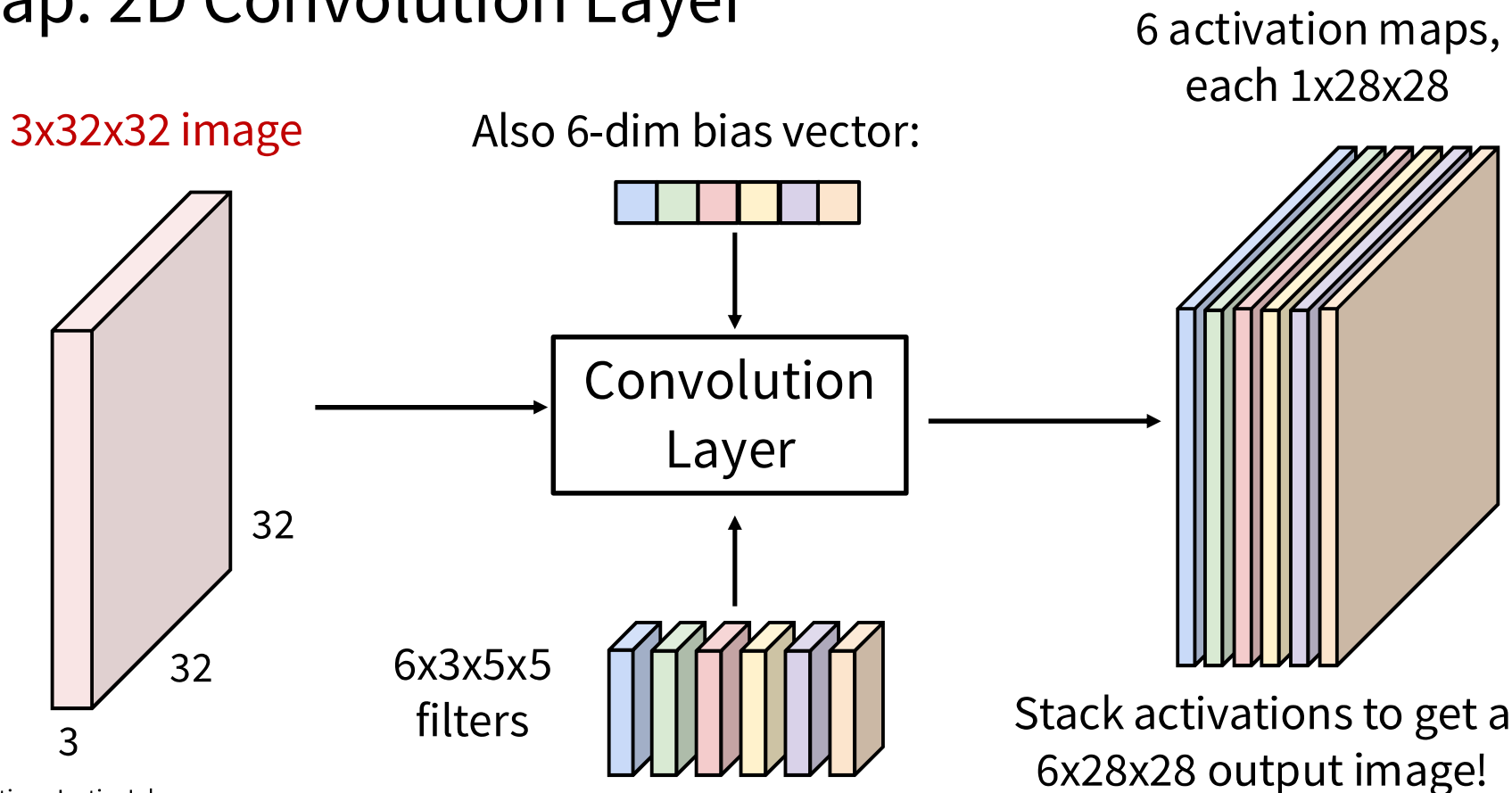



Input:
3 x T x H x W



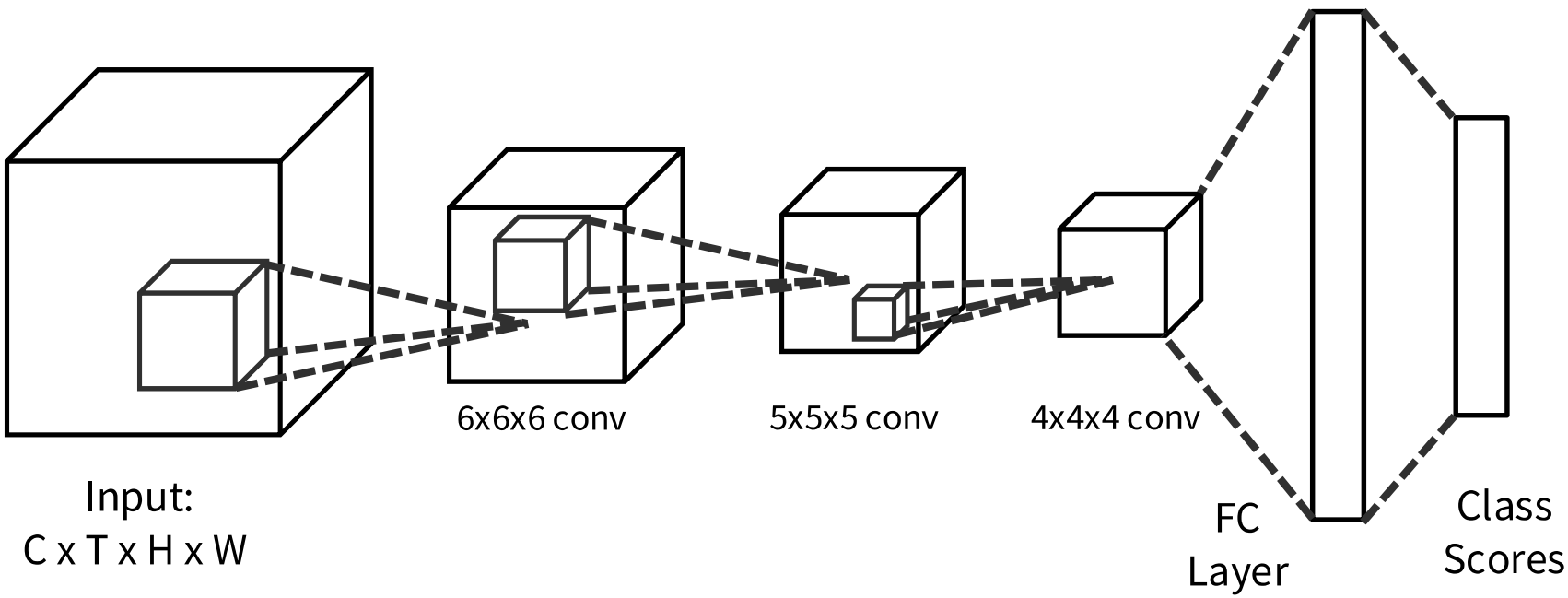
Ji et al, "3D Convolutional Neural Networks for Human Action Recognition", TPAMI 2010; Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

# Recap: 2D Convolution Layer

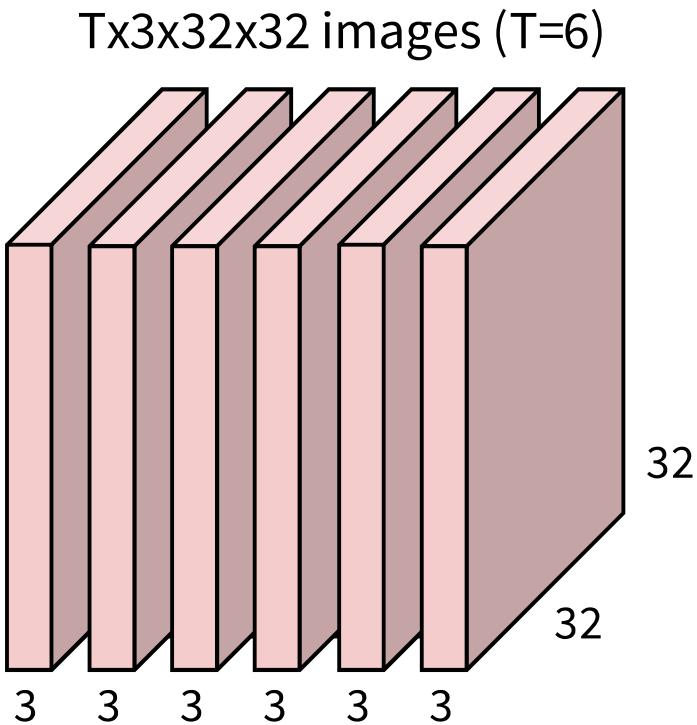



Slide inspiration: Justin Johnson

# 3D Convolution: High Level Idea

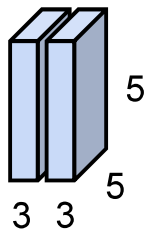


# How is 3D Different? Filters



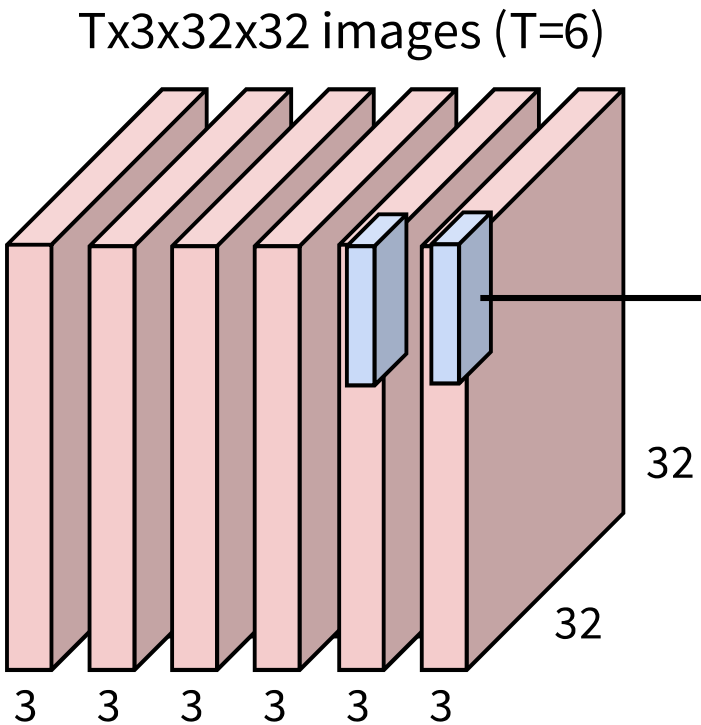

**What does one filter look like?**

One 2x3x5x5 filter




**Now has time dimension!**

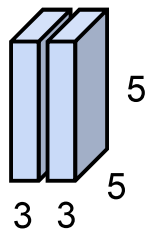
# How is 3D Different? Filters



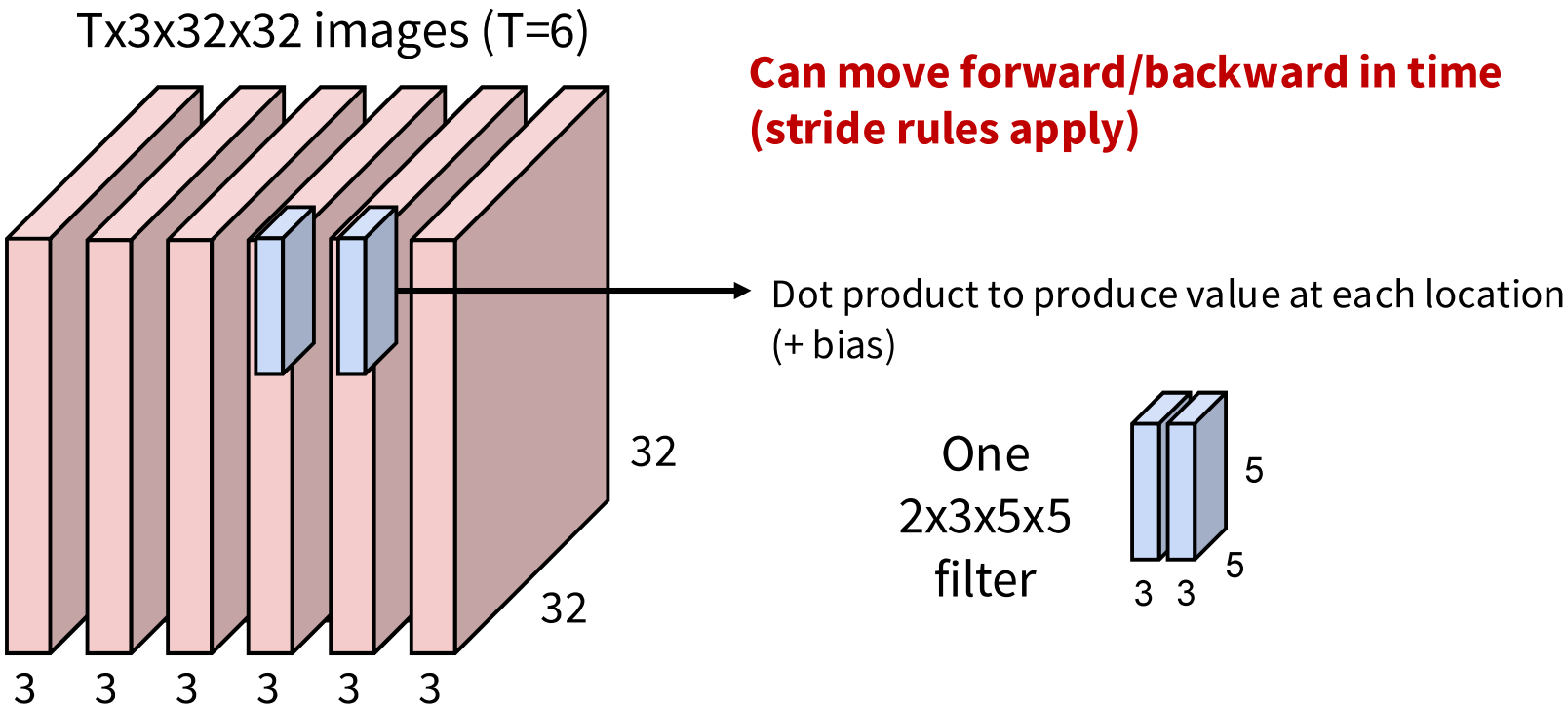

**Sliding window mechanism is the same, with an extra dimension**

Dot product to produce value at each location (+ bias)

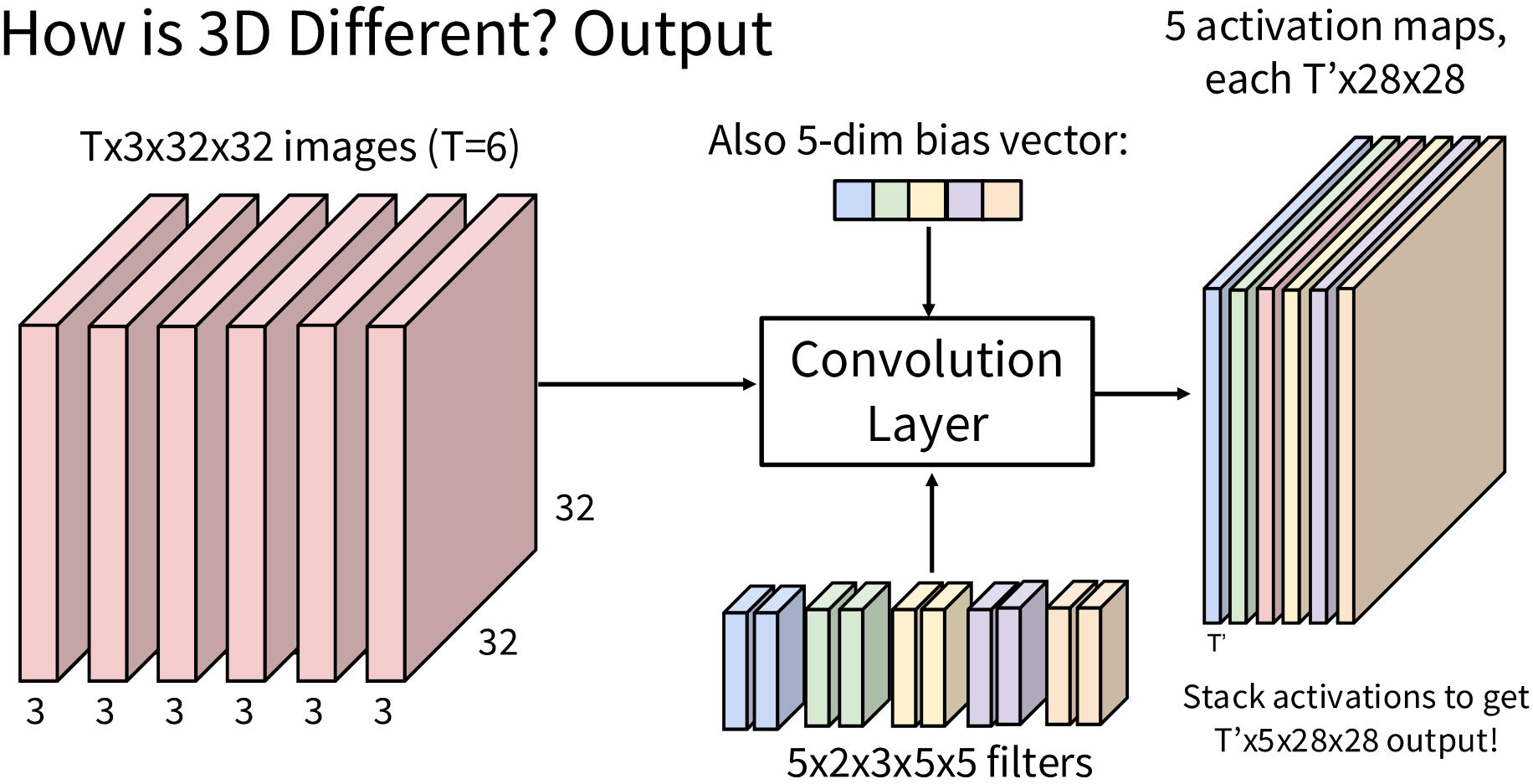
One 2x3x5x5 filter



# How is 3D Different? Stride



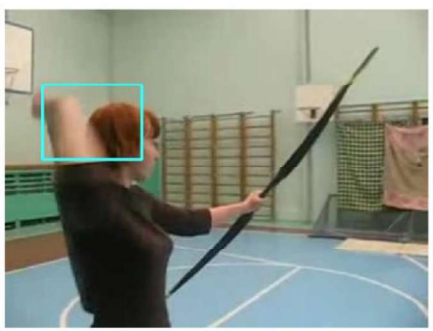
# How is 3D Different? Output



Next: Some Tricks To Improve Performance

# Measuring Motion: Optical Flow

Image at frame t



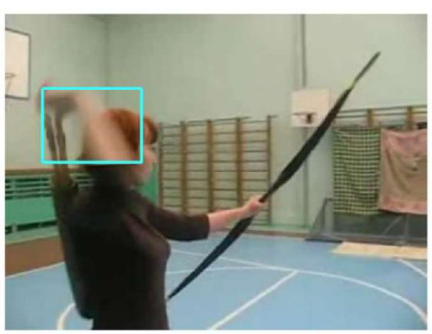
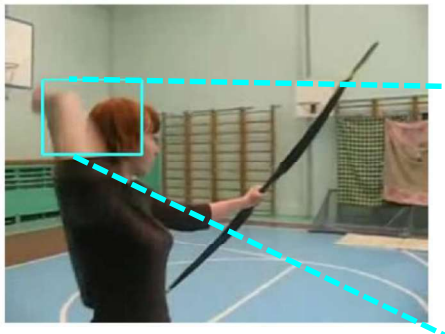


Image at frame t+1

Simonyan and Zisserman, “Two-stream convolutional networks for action recognition in videos”, NeurIPS 2014

# Measuring Motion: Optical Flow

Image at frame t



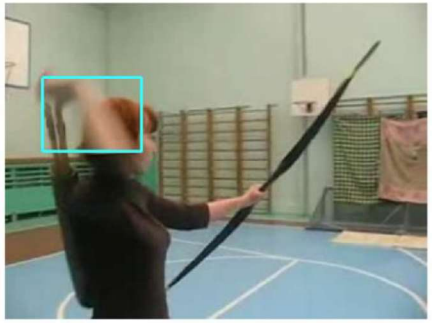
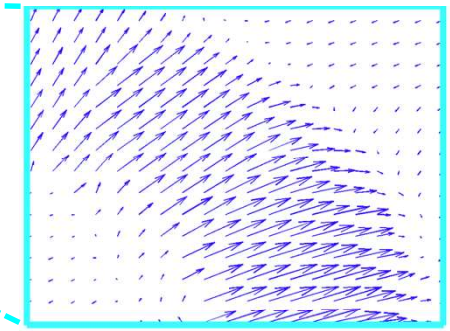


Image at frame t+1

Optical flow gives a displacement field F between images $I_t$ and $I_{t+1}$



Tells where each pixel will move in the next frame:

$F(x, y) = (dx, dy)$

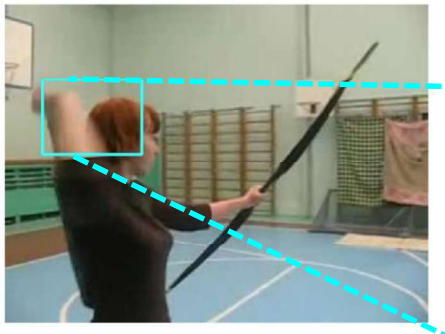
$I_{t+1}(x+dx, y+dy) = I_t(x, y)$

Computed via algorithm or faster NN approximation

Simonyan and Zisserman, "Two-stream convolutional networks for action recognition in videos", NeurIPS 2014

# Measuring Motion: Optical Flow

Image at frame t



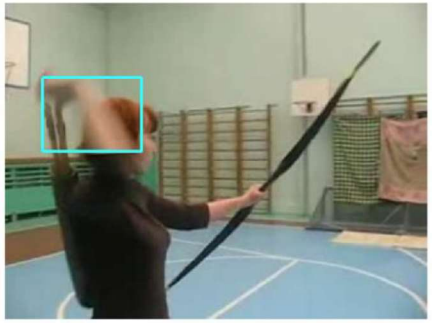
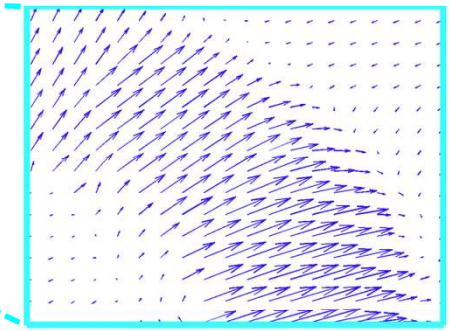


Image at frame t+1

Optical flow gives a displacement field F between images $I_t$ and $I_{t+1}$



Tells where each pixel will move in the next frame:

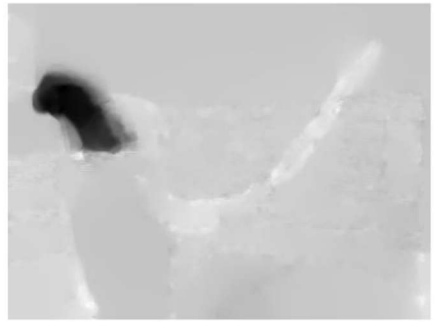
$F(x, y) = (dx, dy)$

$I_{t+1}(x+dx, y+dy) = I_t(x, y)$

Optical Flow highlights local motion

Horizontal flow dx



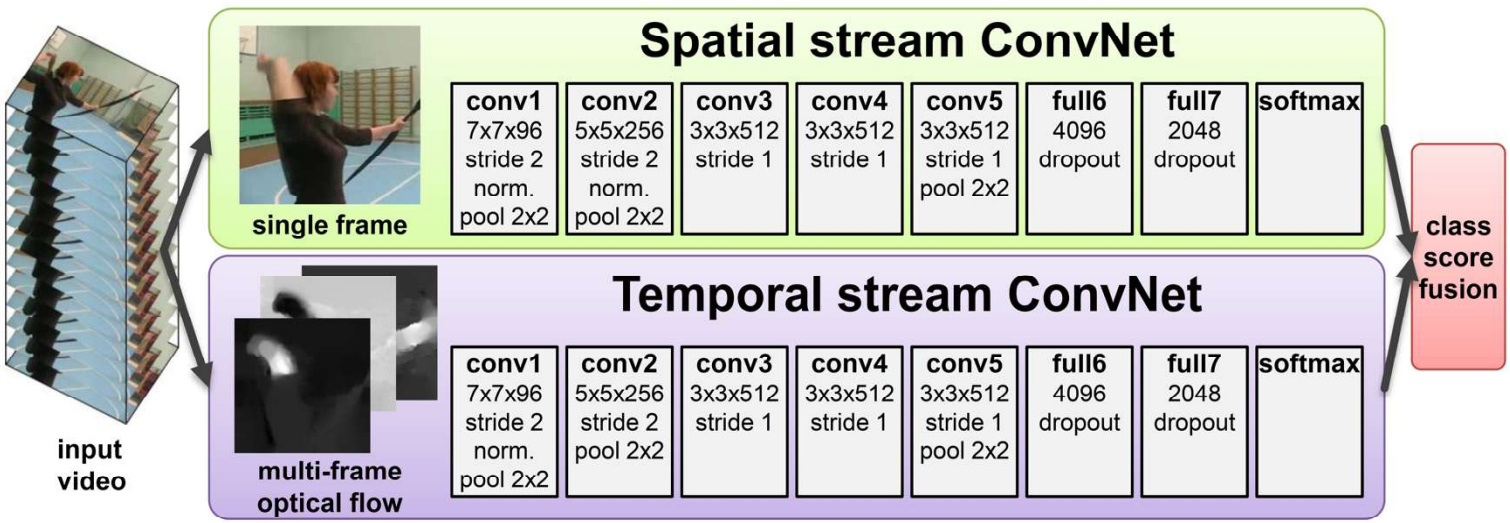


Vertical Flow dy

Simonyan and Zisserman, "Two-stream convolutional networks for action recognition in videos", NeurIPS 2014

# Separating Motion and Appearance: Two-Stream Networks

Input: Single Image
3 x H x W




Input: Stack of optical flow:
[2*(T-1)] x H x W

Early fusion: First 2D conv
processes all flow images

Simonyan and Zisserman, "Two-stream convolutional networks for action recognition in videos", NeurIPS 2014

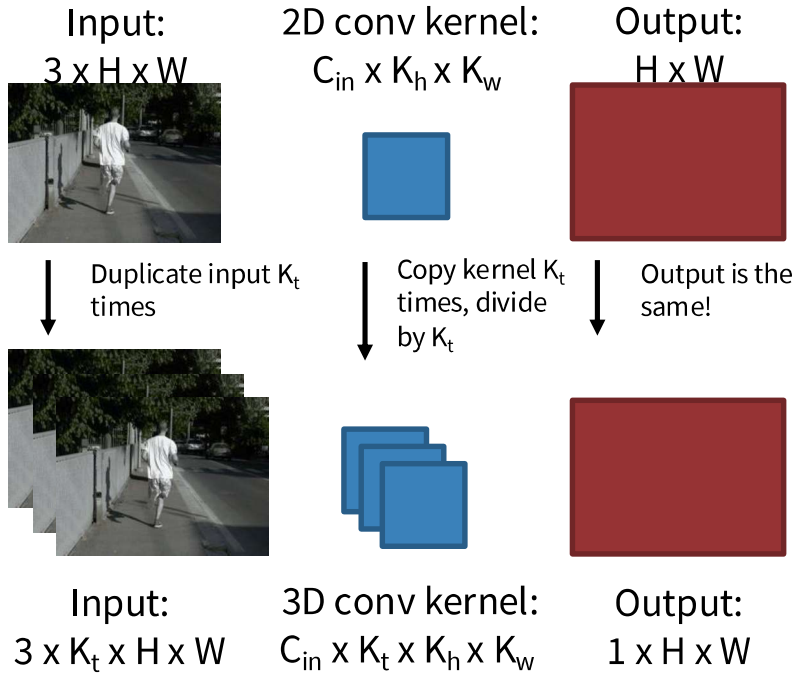
# Inflating 2D Networks to 3D (I3D)

There has been a lot of work on architectures for images.
Can we reuse image architectures for video?

Idea: take a 2D CNN architecture.

Replace each 2D $K_h \times K_w$ conv/pool layer with a 3D $K_t \times K_h \times K_w$ version

Can use weights of 2D conv to initialize 3D conv: copy $K_t$ times in space and divide by $K_t$
This gives the same result as 2D conv given "constant" video input




Carreira and Zisserman, "Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset", CVPR 2017

# Video Understanding: Evolution of the Field

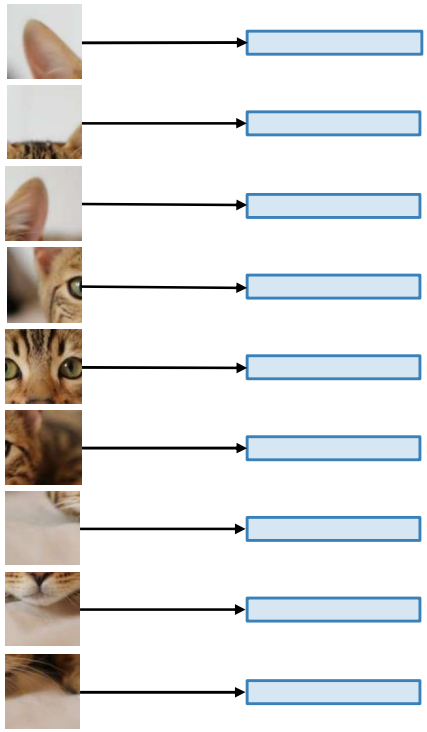
2014 – 2021 Era of 3D CNNs + RNNs

**2021 – 2026 Transformers**

# Image Level: Vision Transformers (ViT)

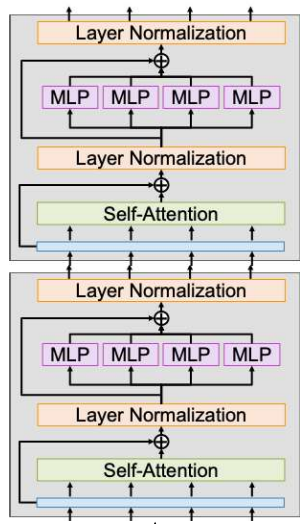


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D




D-dim vector per patch are the input vectors to the Transformer

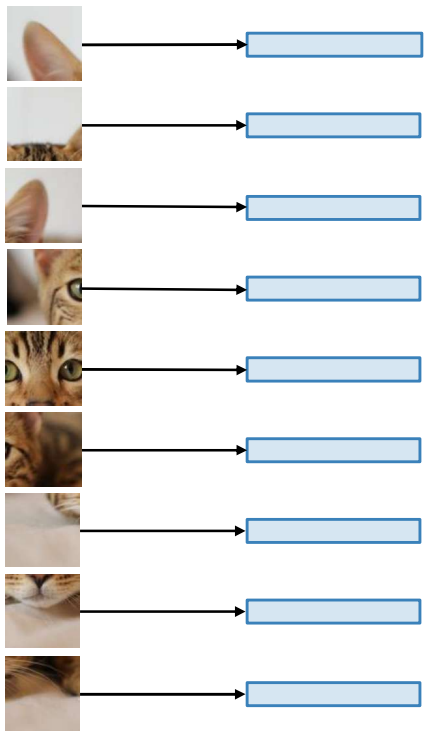
Use positional encoding to tell the transformer the 2D position of each patch

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

# Video Level: Vision Transformers (ViT)



Input Frame:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

14 x 14 = **196 tokens per image**. This is a ton!

**Small clip:** T = 16 → 3136 tokens
**~book chapter**

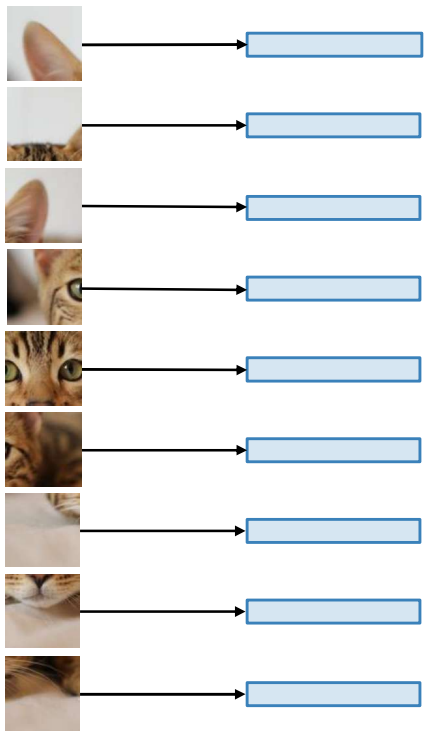
**5 minutes at 1fps:** T = 300 → 58.8k tokens
**~short novel**

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

# Video Level: Vision Transformers (ViT)



Input Frame:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

14 x 14 = **196 tokens per image**. This is a ton!

**Small clip:** T = 16 → 3136 tokens
**~book chapter**

**5 minutes at 1fps:** T = 300 → 58.8k tokens
**~short novel**

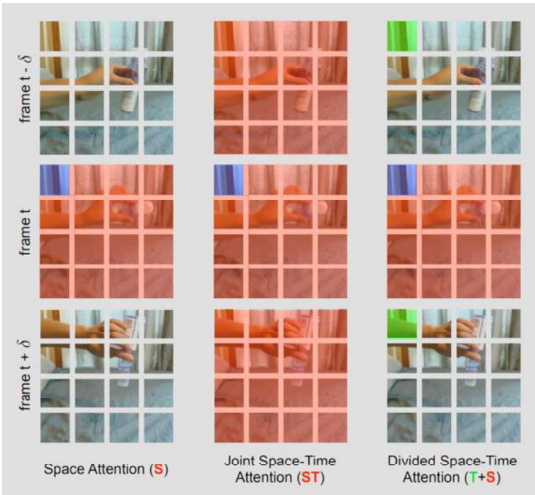
**5 minutes at 24fps** → 1,411,200 tokens
(~context limit of most modern LLMs)

**Naïve patch-level attention scales very poorly! How to solve this?**

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

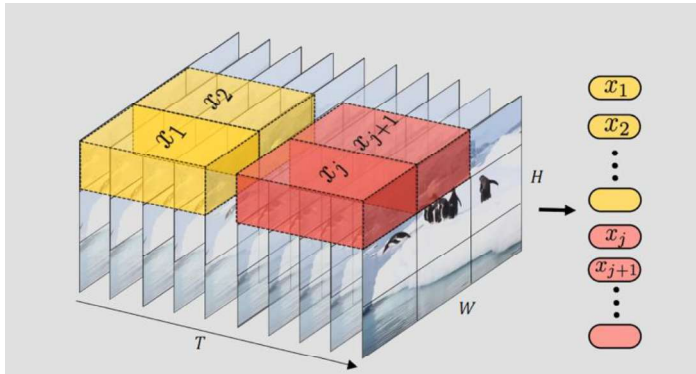
# Two Broad Strategies for Improving Video Efficiency

## Modify attention operator



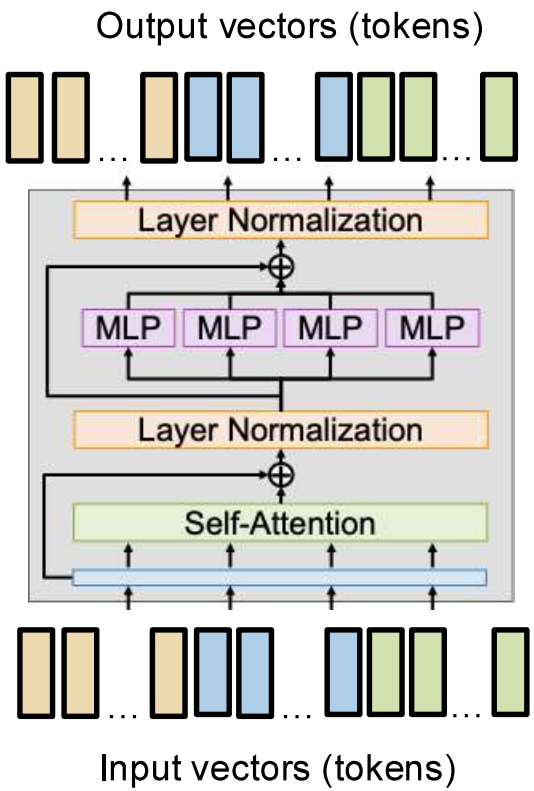

Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021
Liu, Ze, et al. "Video Swin Transformer" CVPR 2022.

## Reduce the number of tokens




Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021
Fan et al, "Multiscale Vision Transformers", ICCV 2021
Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

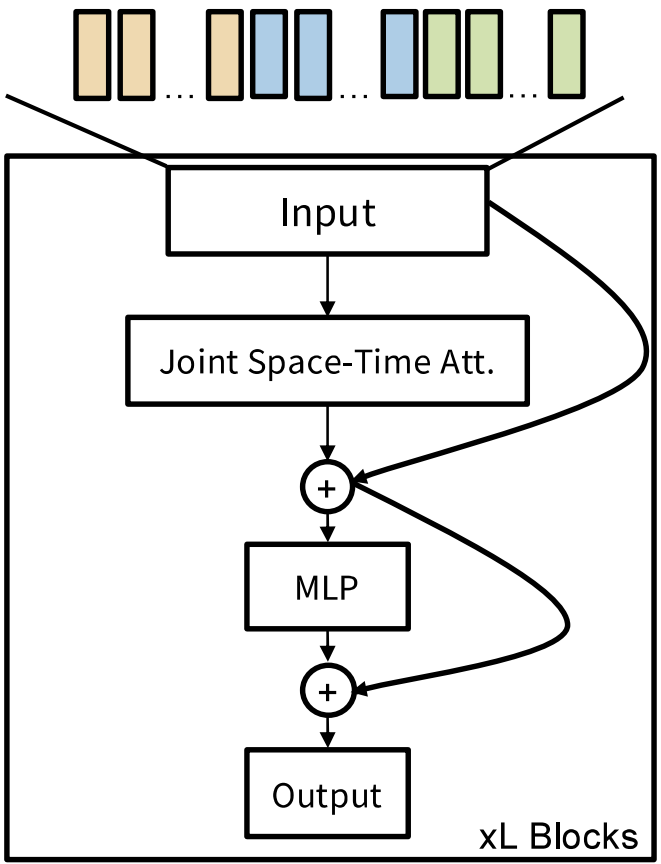
# Recall: Transformer Block




Assume fixed N patches per frame as before (3 shown).

Figures modified from: Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021

# Joint Space-Time Attention



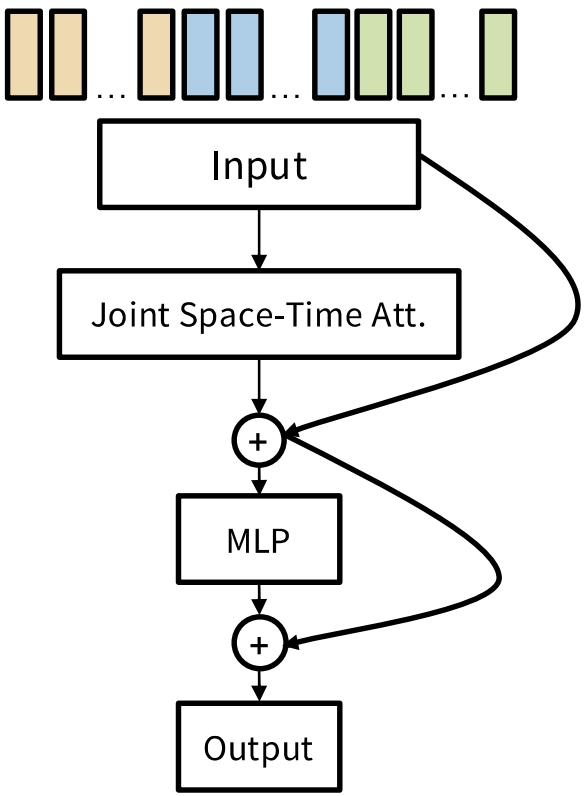

**Extremely inefficient!**

Recall: 5 minutes at 24fps → 1,411,200 tokens

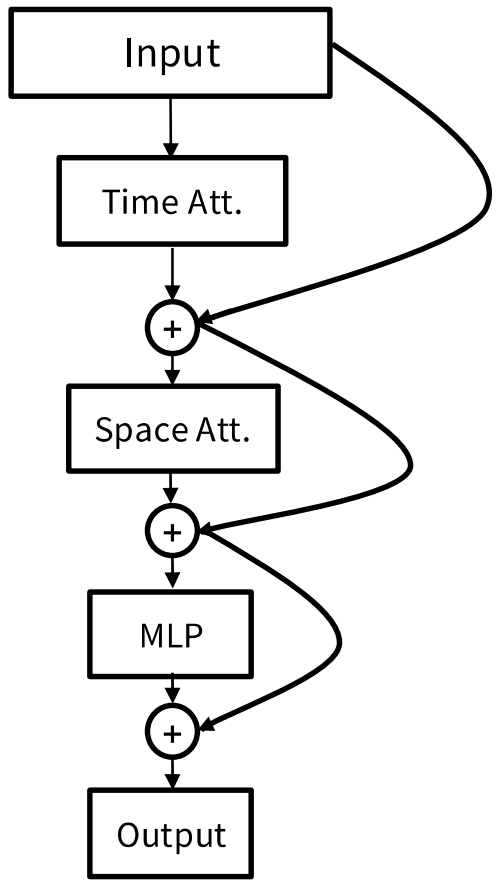
For simplicity, normalization layers not shown in diagram.

Figures modified from: Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021

# Joint vs Divided Space-Time Attention



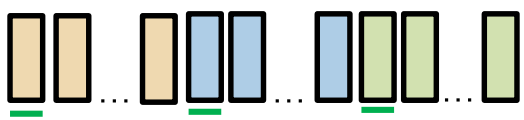

**Key Idea: We can Divide Time and Space Attention Operators into 2 steps**

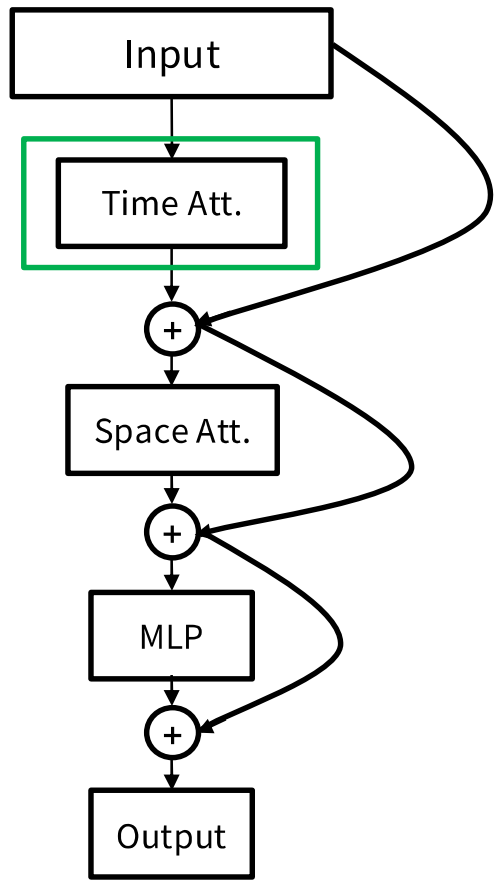


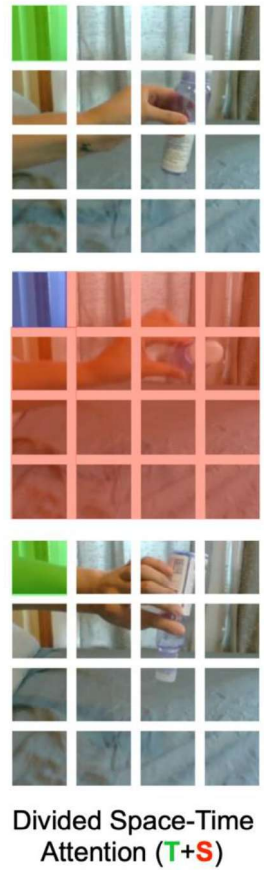

Figures modified from: Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021

# Divided Space-Time Attention

During “Time” Attention Operation, each token attends to the same spatial location across different frames.





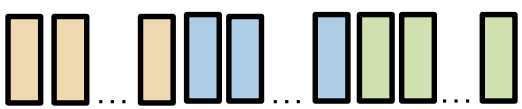



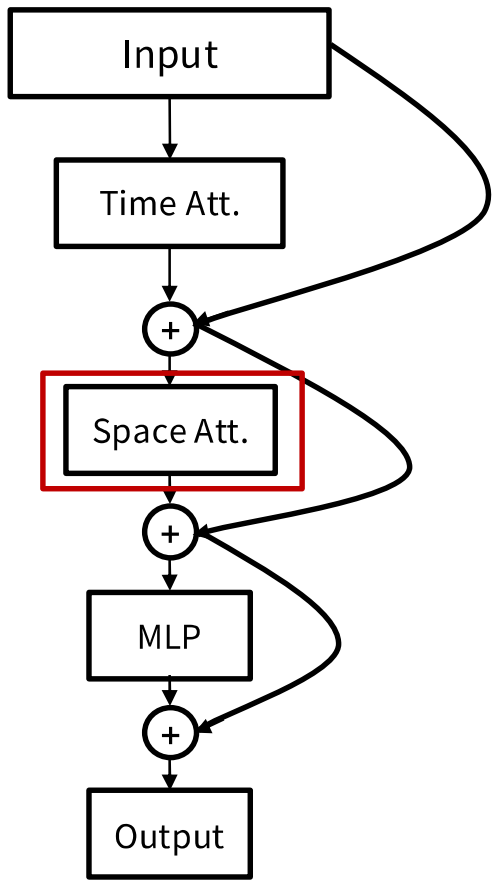
Divided Space-Time Attention (T+S)

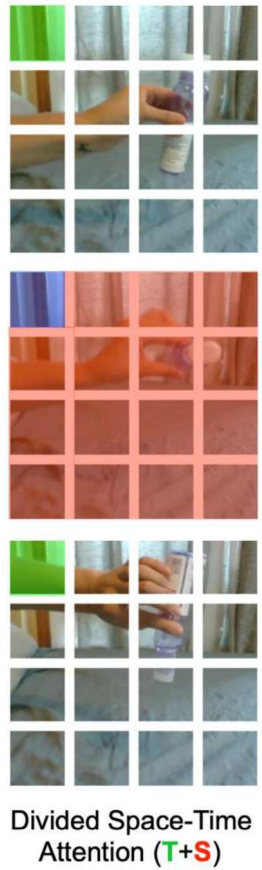
Figures modified from: Bertasius et al, “Is Space-Time Attention All You Need for Video Understanding?”, ICML 2021

# Divided Space-Time Attention

During “Space” Attention Operation, each token attends to other tokens within the same frame





Divided Space-Time Attention (T+S)

Figures modified from: Bertasius et al, “Is Space-Time Attention All You Need for Video Understanding?”, ICML 2021

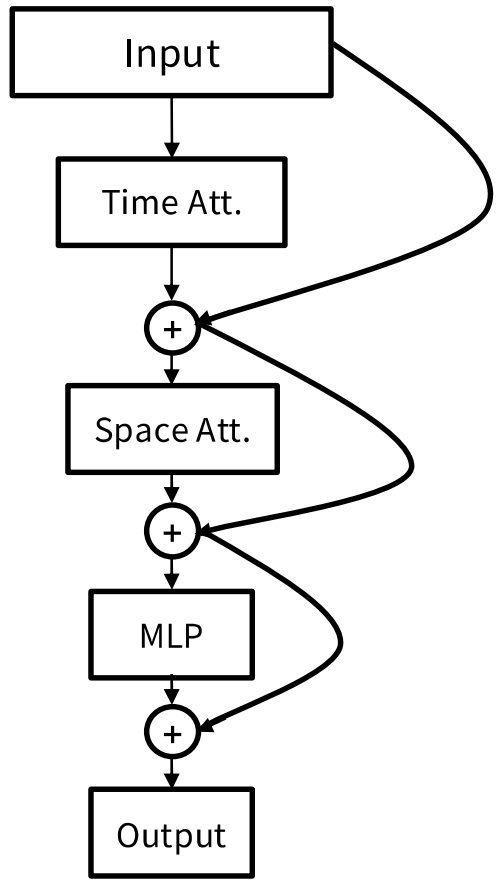
# Divided Space Time Attention

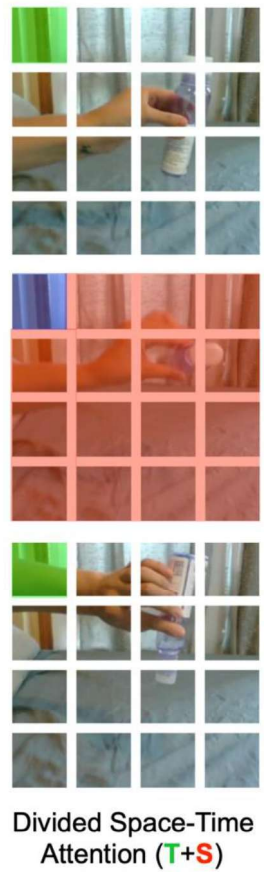
**Why does this work well?**

1. Reduces per-token computational cost from O(NT) to O(N + T)

2. Information can still propagate across space/time after many blocks.





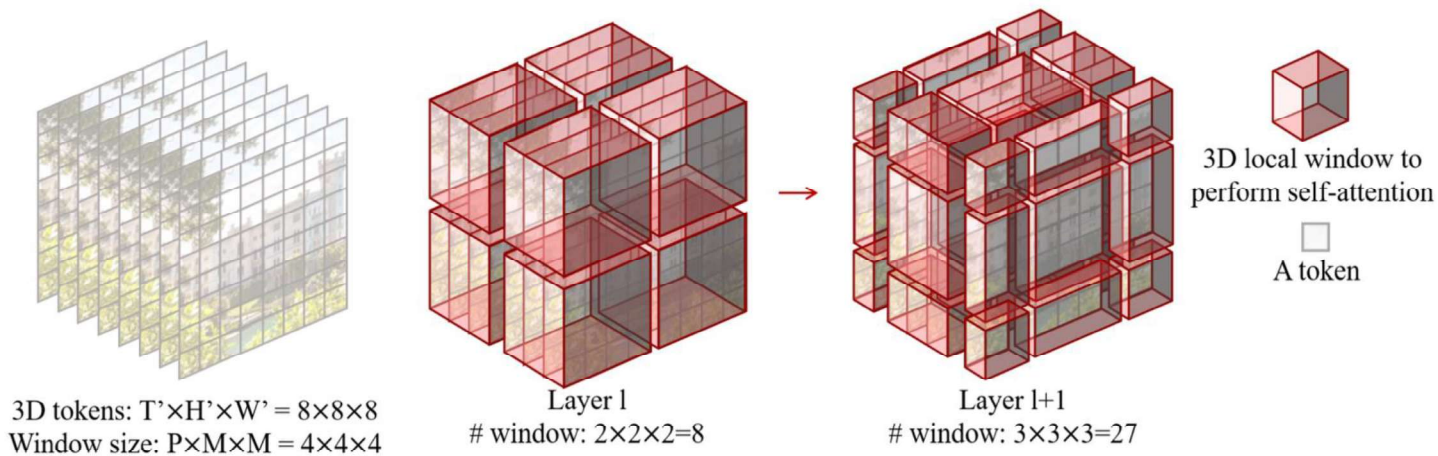
Divided Space-Time Attention (T+S)

Figures modified from: Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021

# Approach #2: Video Swin Transformer

Key idea: Restrict self-attention to small local space-time cubes.
Very similar to 3D CNNs, but with self attention inside each cube.

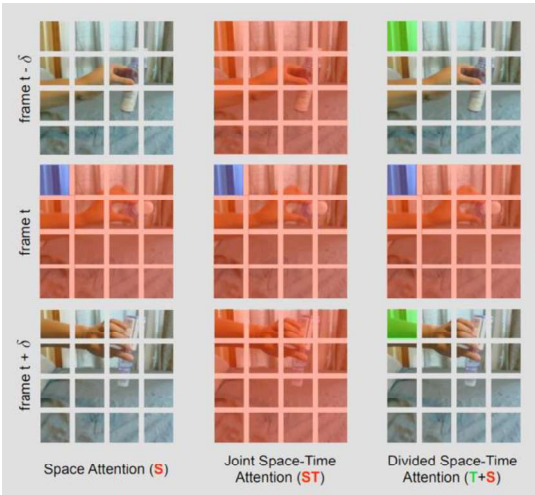
Shift cubes between layers to allow information to pass across cube boundaries.




Figures from: Liu, Ze, et al. "Video Swin Transformer" CVPR 2022.

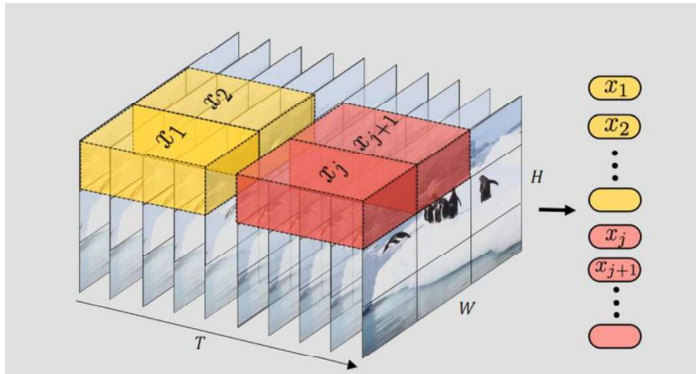
# Two Broad Strategies for Improving Video Efficiency

## Modify attention operator




Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021
Liu, Ze, et al. "Video Swin Transformer" CVPR 2022.

## Reduce the number of tokens




Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021
Fan et al, "Multiscale Vision Transformers", ICCV 2021
Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

# How to reduce the number of tokens?

**One solution: Tubelets**

More efficient than patches!

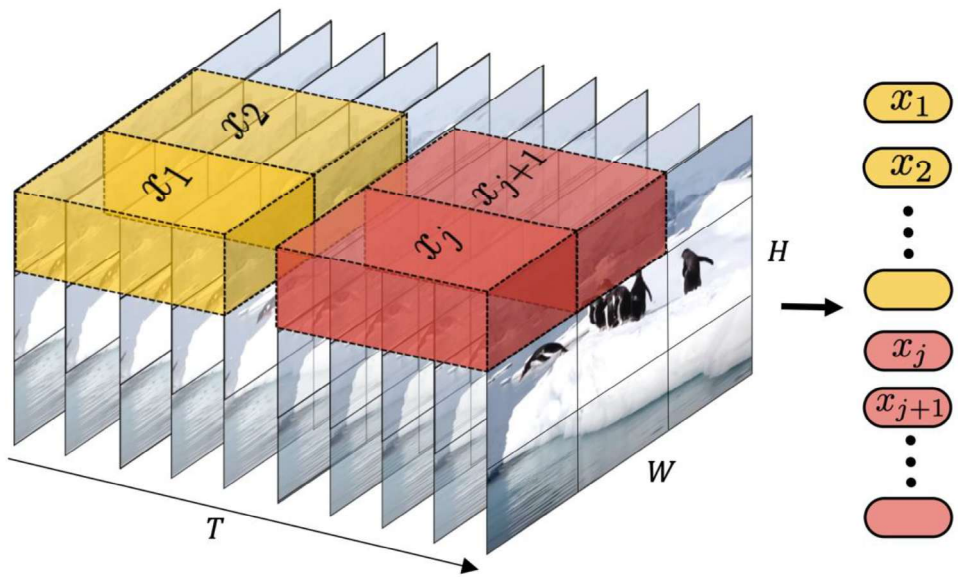



Figure from: Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

# How to reduce the number of tokens?

## One solution: Tubelets

More efficient than patches!

Some intuition:

- Patches do not contain motion information, but tubelets do
- Significantly fewer tokens than patches!

Tubelets over 4 frames:
4x reduction in tokens → 16x reduction in compute

Still project from tubelet dim to the transformer dim.

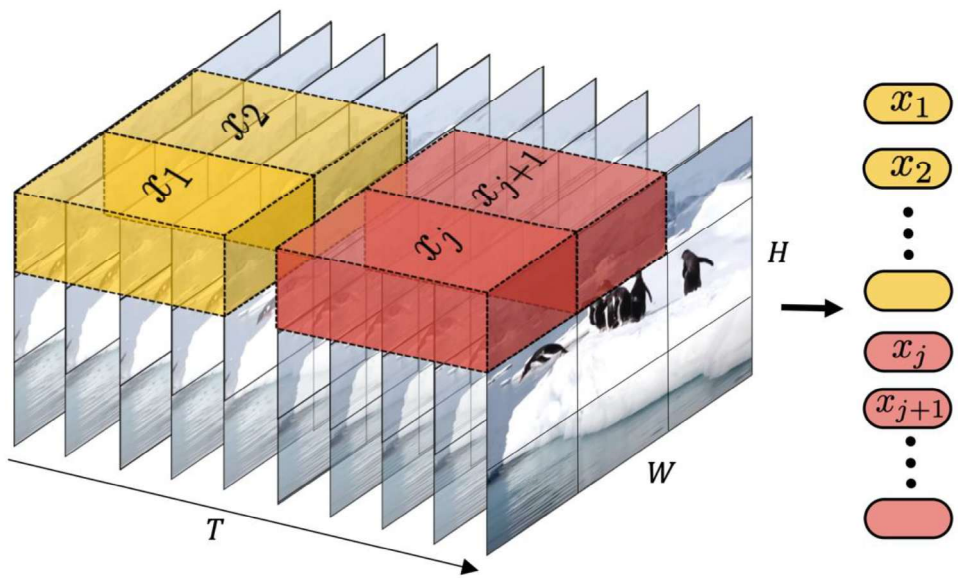



Figure from: Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

# How to reduce the number of tokens?

Employed by *tons* of video understanding transformers (e.g. ViViT, VideoMAE, Video Swin, MViT, V-JEPA)

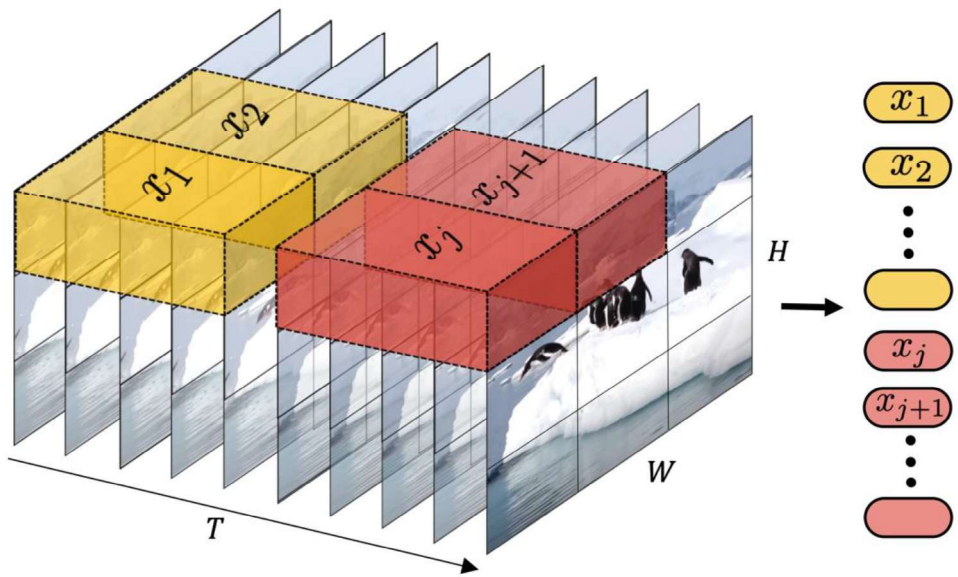



Figure from: Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021